学期助手 · 技术文档

> 第九届传智杯 AI 创新应用挑战赛 · Vibe Coding 全栈应用速成 · 个人赛 · C 组 文档版本：最终版 · 2026-11-10

目录

一、项目与问题定义　1.1 它解决什么问题　1.2 与通用知识库的差异　1.3 交付形态

二、意图控制策略　2.1 控制模型：谁定义、谁实现、谁验收　2.2 约束落在三个层面　2.3 六次真实迭代的完整 Prompt 链　2.4 一次失败的引导　2.5 意图控制的可检验性

三、架构设计　3.1 分层与模块边界　3.2 数据流　3.3 抗幻觉的三层实现　3.4 前端架构与第三方组件的使用边界　3.5 关键决策记录

四、可复现性论证　4.1 环境与依赖　4.2 实测耗时与网络处理　4.3 验证方法　4.4 已知限制

五、总结

一、项目与问题定义

1.1 它解决什么问题

一个把课程资料、课表、作业截止时间放在同一条时间轴上的学习助手。

它做两件事：

1. **回答课程资料里的问题，每个答案都能点回原文页码**；资料里没有答案时明确说找不到，而不是编一个。
2. **告诉你这周有什么课、哪份作业快到期**，把散落在课表和通知里的信息合成一张周视图。

这两个功能看似无关，但共用同一个坐标——**教学周。**资料问答回答「这一章讲什么」， 周视图回答「这一章什么时候学」；两者的结合点，是现有知识库产品没有覆盖的地方。

1.2 与通用知识库的差异

需要诚实说明：RAG 本身是成熟方案，AnythingLLM、RAGFlow、Dify 等产品都做得很完整， 「课程资料喂进去让学生问」也已经有人做。技术层面本项目确实在做别人做过的事。

差异不在技术新颖度，而在数据结构的组合方式：

维度	通用知识库	本项目
时间维度	无	课表、作业、资料共享教学周坐标
回答姿态	被动问答	主动推送「这周该做什么」
不确定时的行为	常常强行作答	三态输出：有依据 / 相关但无答案 / 完全不相关
检索范围	通常不披露	明确告知检索覆盖了哪些文档的哪些页

赛道的评分维度里没有「技术创新性」这一项，命题背景原文是「代码编写成本趋近于零， 考的是定义问题、产品设计、架构编排、指挥 AI」。因此本项目的设计重心放在**可信**与**可控**上， 而不是堆叠检索算法。

1.3 交付形态

部分	技术	说明
后端	Python 3.14 / FastAPI	REST 接口 + SSE 流式
检索	Chroma（本地持久化）+ bge-small-zh-v1.5	512 维，全部本地推理
生成	DeepSeek `deepseek-flash`	OpenAI 兼容协议，温度 0
前端	React 18 + TypeScript + Tailwind	Vite 构建，产物由 FastAPI 直接托管
部署	本地单进程	不做公网部署（赛道不要求在线演示）

全部计算在本机完成，只有用户的问题与检索到的片段会发送给模型服务； 资料原文、向量库、API Key 均不离开本机。

二、意图控制策略

这是本赛道的核心考察项，也是本文档的重点。

2.1 控制模型：谁定义、谁实现、谁验收

整个项目中，「人」和「AI」的分工是被明确约束的：

环节	谁负责	具体表现
定义问题	人	做什么功能、边界在哪、什么算做完
提出约束	人	「扫描件必须报错而不是静默返回空」「页码是一等公民」
提供真实数据	人	真实课表、真实作业节奏、真实 PDF
实现	AI 生成	代码、测试、文档草稿

环节	谁负责	具体表现
验收	人	跑实测、看数据、判断哪里不对
纠偏	人给方向，AI 执行	「92% 断在句中是问题，改切点策略」

关键在两条：

1. **AI 不定义「什么算成功」。**验收标准由人给，且必须是可测量的 （不是「感觉好一点」，而是「句末结尾率从 92% 提升到 80% 以上」）。
2. **AI 的产出必须可验证。**代码要能跑、数据要能复现、结论要能指出证据来源。

2.2 约束落在三个层面

光是「说清楚要求」不够——口头要求会被 AI 的后续修改悄悄推翻。 本项目的做法是把每条重要约束落到**代码、接口、测试**三个层面，让它无法被绕过：

约束	代码层	接口层	测试层
页码可追溯	切片不跨页	每条引用必须带 `page_no`	断言 `page_no >= 1`
不许编造	三层抗幻觉	`status` 只有 ok / not_found / error	资料外提问必须返回 not_found
扫描件不能静默失败	抛 `NoTextLayerError`	上传接口返回 422	——
重复上传幂等	`doc_id` = 内容哈希	——	同字节二次上传，片段数不变
并发不互相干扰	`_ask_lock` 非阻塞获取	429 + 明确文案	并发请求至少一个被拒
旧数据不能丢	启动时补齐 `course` 字段	——	限定课程时未分类资料仍可检索

为什么必须落到三层：开发过程中反复出现的情况是——AI 在一次修改里顺手改坏另一处。 把约束写进测试，才能在下一次改动时立刻发现回归。

2.3 六次真实迭代的完整 Prompt 链

以下六轮都是从真实开发中摘出来的，包括原始提问、AI 的响应、问题的定位过程和我给出的新约束。 更完整的原始对话见 `docs/prompts/raw/` （脱敏导出）。

2.3.1 切片质量：从 92% 断在句中修到 80% 句末结尾

我的初始要求：「按页切，不跨页」——引用要能精确到页码，跨页拼接会让「这个结论在第几页」说不清。

AI 的实现是标准的字符窗口切片：`text[start:start+size]` 配 80 字符重叠，以页为边界。 功能上完全正确，链路跑通。

问题出在验收环节。我拿实际数据验了切分质量：

> 82 个片段中，只有 2 个以句末标点结尾
> → 92% 的片段断在句子中间

用例：「均值是所有数值之和除以个数」这句被切成了「均值是所有数值之和除」+「以个数」。

定位：参数（600 字符 / 80 重叠）本身没问题，问题是**没有把「断点落在哪」当成设计的一部分**—— AI 默认实现的是「按长度切」，而实际需要的是「按语义边界切，长度只是上限」。

我给出的新约束：「切点优先落在句末标点，其次空行，最后才硬切；窗口后 40% 范围内找断点。」

结果：句末结尾率从 8% 提升到 **80%**。剩余 20% 是超长段落（单句超过窗口大小），属于无解情况。

这轮的意义：AI 交付的是「能跑的代码」，而验收标准是「切分质量」。如果我只看了「能跑通」， 这个质量问题会一直留到检索阶段才暴露，而那时候已经很难判断是切片问题还是检索问题。

2.3.2 嵌入模型：0.81 和 0.88 的区别就是没有区别

我的初始要求：「先用 Chroma 默认模型跑通，再评估中文效果。」

AI 用 Chroma 自带的 `all-MiniLM-L6-v2`（英文模型）跑通链路，功能正常。三组查询实测：

查询	最佳距离	是否正确命中
英文问「多少个注意力头」	**0.39**	✅ 命中 p5
中文问同一件事	0.81–0.87	❌ 返回位置编码、参考文献
库外问题（麻婆豆腐）	0.87–0.88	——

判断：中文提问的距离（0.81）和完全不相关的内容（0.88）几乎相同——说明模型对中文没有任何判断力。 课程资料是中文的，必须换。

第二轮的错误：换成 `bge-small-zh-v1.5` 后，中文距离从 0.81 降到 0.58，仍然没命中。 这里我差点误判成「模型还是不行」。

定位：测试设计错了——我用**英文文档 + 中文提问**去考模型，实际场景是**中文资料 + 中文提问**。 这是在考跨语言检索，不是考中文检索。

换成中文样本后重测：第 1 命中就是正确答案，距离 **0.49**。

教训：测试设计本身就是意图的一部分。测试错了，会引导 AI 往错误方向修。

2.3.3 召回不足：正确答案排在第六位

我的约束：「模型可能编造 [7]，而检索结果只有 5 条——引用编号必须逐个校验。」

AI 实现了三层防线，功能正常。但第一次实测出现了误判：

> 问：How many attention heads does the base model use?
> 答：not_found（但最佳距离只有 0.37，很相关）

排查顺序（这一步很重要，不能跳）：

1. 先怀疑 prompt → 不是，模型是**正确地**说「片段里没有」
2. 再看检索结果 → 正确答案（p5，`h = 8 parallel attention layers`）**排在第 6 位**
3. 前 5 名距离 0.373~0.387，第 6 名 0.401——**差距只有 0.03**

结论：`not_found` 有两种原因——「资料里真没有」和「检索没召回」。 不能看到 not_found 就以为抗幻觉生效了。

处理：`top_k` 默认 5 → 8。重测立刻答对，引用指向 p5。

2.3.4 数据建模：作业是稀疏事件，不是周期性条目

我的输入：「作业数据不好造，因为开学前两周好多课程都没作业。」

AI 最初的方案是给每门课按周排作业——这是一个整齐但错误的模型假设。 真实情况是：作业有就有、没有就没有，且分布极不均匀。

改为：`{id, course, title, due_date}` 的一维事件表，到期才出现在视图里。

同一轮里还有两个纠偏：

- **课名不能按字面理解**：AI 把「程序设计基础」当成编程课，实际网页制作课在课表里就叫这个名字。
- **砍掉数据不支持的功能**：曾计划支持「第三章是哪周学的」，查课表结构后发现只有 「课程 / 周几 / 第几节 / 哪几周上」，没有任何教学进度信息。两条出路（人工标注 / 放弃），选择放弃。

教训：功能取舍要问「数据支不支持」，不问「看起来酷不酷」。

2.3.5 判定标准过严：能答的问题被整体拒答

怎么发现的：给作品文档拍截图时，随手问了一句 「均值、中位数、众数、标准差分别是什么，各自适合什么场景？」 ——这是资料里四个定义都写全了的问题，系统却返回「里面没有直接回答这个问题的信息」。

先分清楚是检索问题还是生成问题（这一步决定修哪里，不能跳）：

检查项	结果
召回条数	3 条，三个片段全进了上下文（920 字符）
最佳距离	**0.287**（远低于阈值 0.65，非常相关）
阈值拦截	未触发

→ 不是检索问题，是模型判断拒答。

根因：prompt 第 3 条是二元的——

> 如果片段中没有能回答问题的信息，只输出 NO_ANSWER 四个字母。

问的是「四个定义 + 各自适合什么场景」。片段里四个定义都有，但「场景」只覆盖了一部分。 模型按「没有完整答案」处理，整条拒答。

修复：把判定从二元改成「能答多少答多少 + 明说缺什么」：

> 只要片段里包含问题**任何一部分**的答案，就把能答的部分如实回答；资料没有涉及的方面， 在回答末尾单独用一行写「资料未覆盖：<具体缺哪个方面>」。 只有当片段与问题**完全无关**、一点相关信息都没有时，才只输出 NO_ANSWER。

验证（关键：确认底线没被削弱）：

用例	改前	改后
四个指标分别是什么 + 适用场景	not_found（整体拒答）	**ok，四个定义全部答出**
太阳系有几颗行星（资料外）	not_found	**not_found**（底线未退化）
均值是什么（简单问题）	ok	ok
这份资料讲了什么（全局概括）	ok	ok，三段 + 3 条引用

回应 2.3.3 那句「not_found 有两种原因」：这次是第三种——**判定标准过严。** 三种原因对应的修法完全不同：资料真空 → 无解；召回不足 → 调 top_k；标准过严 → 调 prompt。 统一当成「抗幻觉生效」就会一直误判。

2.3.6 前端全量迁移：一天内完成，靠的是把拆解做在前面

这是最后一次、也是规模最大的一次 AI 协同。

背景：初版前端是 Streamlit（单文件，用于快速验证闭环），决赛版需要换成 React。

我的关键决策：先让 AI 把工作量拆开算清楚，再决定做不做。

AI 给出的拆解：

阶段	内容	工时
后端 API 补全	从只有 `/health` 到完整的 REST 层	1 天
流式输出	SSE	0.5 天

阶段	内容	工时
前端页面	上传、对话、出处卡片、周视图、导入面板、设置	3–4 天
联调修 bug	CORS、状态管理、构建打包	1–2 天
重录演示视频	界面全变	1 天
合计		**6.5–8.5 天**

优化拆解（我要求的第二版）：找现成的组件库、用 Vite 不用 Next.js、后端先 mock 前端并行开发、 直接拷组件源码而不是装包、视频文档只改增量——合计压到 **7.75 天。**

实际执行：一天内完成前后端主体迁移 + 界面 + 组件整合，第二天完成审查、测试和修复。

为什么能压缩这么多：不是 AI 写得更快，而是**拆解做得足够细**—— 每一小块都是「输入明确、输出可验证」的任务，没有含糊的「把前端迁移一下」。 拆解本身就是意图控制的产物。

2.4 一次失败的引导

诚实记录一次没做好的地方。

场景：前端迁移完成后，我让 AI「整体检查前端效果」。AI 自己看不出问题， 于是我改为派一个独立的子 agent 做评测——只给任务描述，不给我的判断。

结果：子 agent 报出 19 个问题，其中 3 个是 P0 级真实缺陷：

1. 「找不到答案」时同一段文字被渲染两次（AI 自己写的 bug）
2. 滚动到底部差 176px，最后一张出处卡片被悬浮输入框盖住
3. 超长无空格字符串溢出气泡

为什么自己看不出来：开发时的注意力在「功能是否跑通」，而评测需要的是「使用者视角」。 同一个人（或同一个上下文）很难同时扮演两个角色。

得到的做法：重要交付前，另起一个不带先入判断的评测者。这不是 AI 特有的问题， 人类开发也一样——所以代码审查才需要独立的人。

2.5 意图控制的可检验性

意图控制不是口号，可以落到可检验的标准上：

可检验项	本项目的做法
每条约束有没有落地	见 2.2 的三层表格，每条都能指到具体代码 / 接口 / 测试
AI 的产出有没有被验证	所有结论都附带实测数据（距离值、耗时、通过率）
失败有没有被记录	`docs/prompts/` 下四份完整记录，含错误路径
决策有没有依据	`docs/决策记录.md` 十条 ADR，每条写明理由与代价

可检验项	本项目的做法
演进链是否真实	Git 提交细粒度，每次提交对应一次真实迭代

三、架构设计

3.1 分层与模块边界

```
┌──────────────────────────────────────────────────────────┐
│  前端  React 18 + TypeScript + Tailwind                  │
│  资料问答 / 周视图 / 资料库 / 课表导入 / 模型配置          │
└──────────────────────┬───────────────────────────────────┘
                       │ REST + SSE
┌──────────────────────▼───────────────────────────────────┐
│  接口层  api.py                                          │
│  14 个端点；只做路由与序列化，不含业务规则                 │
└──────────────────────┬───────────────────────────────────┘
                       │
┌──────────────────────▼───────────────────────────────────┐
│  业务层                                                  │
│  parser ── 解析 PDF，无文本层报错                        │
│  chunker ─ 按页切片，句末优先                            │
│  embedding ─ BGE 中文模型，查询侧加指令前缀              │
│  store ──── Chroma 读写，课程过滤                        │
│  qa ─────── 检索编排、三层抗幻觉、引用校验               │
│  semester ─ 周次计算、稀疏作业                           │
│  render ─── PDF 页渲染为 PNG                             │
└──────────────────────┬───────────────────────────────────┘
                       │
┌──────────────────────▼───────────────────────────────────┐
│  数据   chroma_data/（向量）· backend/data/pdfs/（原件留底）│
│         local/data/（用户课表作业，不进仓库）             │
└──────────────────────────────────────────────────────────┘
```

分层原则：接口层不含业务规则。判断「资料里有没有答案」是 `qa` 层的职责， `api` 只负责把结果序列化。这样命令行工具（`backend/cli.py`）可以复用同一套业务逻辑， 不需要重新实现一遍判定。

3.2 数据流

入库:

```
PDF ──parse──→ 逐页文本 ──chunk──→ 带页码的片段
                                        │
                    ┌───────────────────┘
                    ▼
```

```
            原件复制留底（doc_id.pdf）
                  │
                  ▼
            BGE 嵌入 → Chroma（metadata 含 doc_id / page_no / course）
```

问答：

```
问题 ──嵌入──→ 向量检索（可按课程过滤）
                  │
                  ▼
          最佳距离 > 0.65?
            ├─ 是 → not_found（不调模型，省成本也杜绝编造）
            └─ 否
                  ▼
          同文档优先筛选（减少跨资料噪声）
                  ▼
          组装带 [n] 编号的上下文
                  ▼
          DeepSeek 生成（温度 0，流式）
                  ▼
          正则校验 [n] 是否真实存在
            ├─ 全部越界 → 判定编造，降级 not_found
            ├─ 部分越界 → 只保留有效的
            └─ 无引用 → 模型未守规则，降级 not_found
                  ▼
          返回（答案 + 引用 + 检索范围 + 耗时）
```

3.3 抗幻觉的三层实现

层	机制	实现位置	实测数据
检索层	最佳距离 > 0.65 直接返回，不调模型	`qa.answer()`	库外问题「麻婆豆腐」= 0.6515，被拦下
提示层	system prompt 硬约束：只用片段、标编号、无依据输出 NO_ANSWER	`qa.SYSTEM_PROMPT`	——
后处理层	正则校验引用编号，全部越界即降级	`qa.answer()`	模型引用 `[7]` 而只检索到 5 条时触发

为什么第三层必须有：prompt 是建议，不是保证。模型完全可能输出一个格式正确但指向不存在的引用。只有后端做后处理校验，才能把「模型守规矩」变成「系统保证」。

第三层的实现（`backend/app/qa.py`，节选）：

```
# 检索层：相关性太差就不调模型
best = min(h.distance for h in hits)
if best > MAX_DISTANCE:          # MAX_DISTANCE = 0.65
    return _not_found(hits, best)
```

```
# 提示层：system prompt 硬约束
SYSTEM_PROMPT = """你是一个严谨的课程资料助手。你必须严格依据用户提供的资料片段回答。
规则：
1. 只能使用下面编号片段里的内容作答。
2. 每一条结论后面都要标注来源编号，格式为 [1]、[2]。
3. 如果片段中没有能回答问题的信息，只输出 NO_ANSWER 四个字母。
4. 不得编造页码、作者、数字、公式。"""

# 后处理层：模型说的 [n] 必须真实存在
used = {int(n) for n in re.findall(r"\[(\d+)\]", raw)}
valid = {i for i in used if 1 <= i <= len(hits)}
if (used and not valid) or not valid:
    return _not_found(hits, best)   # 引用全越界 / 一个都没有 → 判定不可信
```

三层的顺序是有讲究的：**先用最便宜的方式拦住**（算个距离就返回，不花钱），**再约束**（prompt），**最后兜底**（正则校验）。如果反过来——先问模型再判断——成本会高一个数量级，而且模型已经生成的内容会诱导人去相信它。

三态输出（而不是简单的有/无）：

状态	触发条件	用户看到什么
ok	有答案且有有效引用	答案 + 可展开的出处 + 原页
not_found（相关但无答案）	检索到了相关内容，但模型判断片段里没有直接答案	「检索到了《X》第 N 页，但里面没有直接回答这个问题的信息」+ 最接近的原文
not_found（完全不相关）	最佳距离超阈值 / 库为空	「没有检索到」+ 已检索的范围

3.4 前端架构与第三方组件的使用边界

技术选型：Vite + React 18 + TypeScript + Tailwind，不用 Next.js （本项目不需要 SSR 和 Node 后端，Next.js 只增加构建复杂度）。

第三方组件：使用了 open-claude-ui（MIT）的部分组件：`ConfirmationDialog`、`CdsButton`、`ScrollFadeContainer`、`StatusIndicators` 以及 183 个设计令牌。

使用边界（这是必须说清楚的部分）：

用了什么	怎么用的
组件结构	按原样引入，改 import 路径
设计令牌（灰阶梯度、圆角尺度、阴影层次）	引入 `cds-theme.css`，覆盖表面色为暖白体系
配色	**替换**：原项目主色是陶土橙 `#d97757`（Claude 桌面端品牌色），本项目整体映射为靛蓝 `#3f61a3`

用了什么	怎么用的
视觉语言	**不复制**：不还原 Claude 的布局与排版，只借用令牌体系与动效参数

为什么不直接照搬：该组件库是高保真还原 Claude 桌面端的产物。 如果作品外观与 Claude 一致，评委会判定缺乏原创性。因此配色、字体、间距全部按本项目定位重新定义， 只保留「结构化的灰阶梯度 + 克制的阴影层次」这套工程方法。

`README.md` 中保留了完整的第三方组件声明与 MIT 版权说明。

3.5 关键决策记录

项目全程维护了一份 ADR（`docs/决策记录.md`），每条记录理由与代价。摘录：

决策	理由	代价
选 Vibe Coding 赛道而非 LLM 科研助手	后者要求六大模块 + 官方测试集 + Docker + 公网演示，是团队三个月工程量	放弃了技术深度更高的赛道
不做多 Agent 架构	单人 26 天，多 Agent 容易变成一堆半成品互相超时	架构图不够「壮观」
不部署公网	赛道不要求在线演示	——
切片不跨页	页码是唯一可回溯的坐标	牺牲一点上下文连续性
嵌入模型必须按语言选	英文模型对中文无判断力（实测）	模型体积增加约 100 MB
砍掉「第几周学哪章」	课表无教学进度数据	少一个演示亮点
删除 Streamlit 界面	早期验证用，不进最终交付	失去一条「一条命令看效果」的路径

决策记录的用途：新会话接手时不需要重新论证已有结论，避免反复在同一个问题上打转。

四、可复现性论证

4.1 环境与依赖

项	版本	说明
Python	3.12+（实测 3.14.10）	`pyproject.toml` 要求 `>=3.12`
uv	0.13.7	依赖管理
Node.js	18+	仅构建前端需要
主要库	FastAPI 0.162 / Chroma 1.5.9 / PyMuPDF 1.28.5 / React 18	完整清单见 `pyproject.toml` 与 `frontend/package.json`

一条命令启动：`.\setup.ps1`（装依赖 → 构建前端 → 启动服务）。 也可手动四步，见 `README.md`。

4.2 实测耗时与网络处理

冷启动完整流程（同一台机器实测）：

阶段	耗时	说明
`uv sync`（无缓存）	**311 秒** / 1.29 GB	含 PyTorch
嵌入模型下载	**104 秒** / 约 100 MB	走 hf-mirror.com
前端 `npm install` + `build`	约 60 秒	走 npmmirror
合计	**约 7 分钟**	满足「十分钟极速复现」

网络问题的处理（都是实测定下来的，不是猜的）：

问题	实测	处理
官方 PyPI 的 wheel 下载 403	多次复现	`setup.ps1` 固化华为云镜像（实测 7.5 MB/s）
清华 TUNA 源本机不可用	HTTP 000	不用
HuggingFace 直连失败	——	代码里默认 `HF_ENDPOINT=https://hf-mirror.com`
模型已缓存时联网检查拖慢启动	——	检测到本地缓存即设 `HF_HUB_OFFLINE=1`

4.3 验证方法

两条冒烟测试，覆盖后端接口与前端交互：

```
# 后端：20 项，约 10 秒
uv run pytest tests/test_smoke.py -v

# 前端：14 项，Playwright 驱动真实浏览器
python -X utf8 tests/ui_smoke.py
```

后端测试覆盖： 健康检查、设置接口不回传密钥、资料库结构、上传入库、幂等性、原页 PNG、越界 404、 非法 doc_id 拒绝、未知接口返回 JSON 404、带引用回答、拒答资料外内容、SSE 事件、 并发守卫、周视图结构、课表导入校验、课程标签与过滤、删除无残留。

测试不依赖用户数据：测试用的 PDF 由 PyMuPDF 现场生成、用完删除； 问答部分幂等地上传仓库自带的示例讲义（内容哈希固定，不产生副本）。

前端测试覆盖：页面加载、提问 → 流式 → 出处展开、拒答、周视图 7 列、 导入面板、1280 宽无横向溢出。

另外做过一次「从零」验证：把仓库中仅版本库追踪的文件复制到干净目录（等同于 clone）， 交给一个不了解开发过程的独立评测者，严格按 README 逐步操作。结果是 11 项功能全部跑通， 同时暴露出 README 的三处描述与代码不符（其中一处是 Streamlit 时代遗留的界面文案）、 测试脚本端口硬编码、以及 `/health` 的配置检测只读 `.env` 等问题，均已修正。

这件事本身说明：**「我认为能跑」和「别人拿到能跑」是两回事。**自测时人会下意识跳过 已经知道的步骤，只有让不知情的人照着文档走，才会发现哪一步其实依赖了文档外的知识。

4.4 已知限制

诚实列出：

限制	原因	影响
扫描件无法识别	只做文本层提取，不做 OCR	上传扫描件会明确报错，不会静默入库
复合问题只答能答的部分	资料没覆盖的方面会标「资料未覆盖」，而不是硬编	需要用户理解这是「部分回答」而非「完整答案」
首字 25 秒超时未实测	无法构造「模型 25 秒不吐字」的场景	代码逻辑有，但未真实触发验证
同一时刻只能处理一个问答	并发会同时调用模型、浪费成本	并发请求返回 429 并提示等待
只支持 PDF	——	——

五、总结

这个项目的核心不是 RAG——那是成熟技术。核心是**如何在 26 天里，用 AI 把一个真实需求做成可交付的产品， 并且每一步都能说清楚「为什么这么做」。**

具体体现在三件事上：

1. **每条约束都落到可执行的位置。**不是「要注意页码准确」，而是「切片不跨页 + 接口必须带 page_no + 测试断言 page_no ≥ 1」。
2. **每个结论都有实测数据。**92% 断在句中、0.81 与 0.88 没有区别、正确答案排在第 6 位而差距只有 0.03—— 这些数字不是装饰，是决定下一步做什么的依据。
3. **失败和误判都被记录。**测试设计错误（拿英文文档考中文模型）、自己写出的双重渲染 bug、 自己看不出问题而必须另起评测者——这些比「成功经验」更能说明工作方式。

AI 在这个过程中扮演的是「执行速度极快的实现者」，而定义问题、设定约束、验收结果、 判断哪里不对——这些始终在人这边。这就是赛题所说的「AI 编写代码，人类引导意图」。

附录 A：模块接口速查

A.1 后端模块

模块	核心接口
`config.py`	`ROOT` / `CHROMA_DIR` / `PDF_STORE_DIR` / `SEMESTER_START`
`parser.py`	`parse_pdf(path) -> ParsedDoc{doc_id, filename, pages[], blank_pages[]}`；`NoTextLayerError`
`chunker.py`	`chunk_doc(doc, size=600, overlap=80) -> list[Chunk]`；不跨页、句末优先
`embedding.py`	`BGEEmbeddingFunction`（`embed_documents` / `embed_query` 分离实现）
`store.py`	`search(q, top_k, course)` / `upsert` / `replace_chunks` / `purge_document` / `docs`
`qa.py`	`answer(q, top_k=8, store, course) -> Answer`； `answer_stream(...)` 逐事件 yield
`semester.py`	`week_no(d)` / `week_view(d)` / `load_courses` / `validate_courses` / `reload_data`
`render.py`	`render_page(pdf, page_no, dpi) -> PNG bytes`； `page_count(pdf)`
`settings.py`	`load()` / `save()` / `clear()` —— 优先级：界面配置 > `.env` > 默认值

A.2 REST 端点

方法	路径	说明
GET	`/health`	存活与 key 状态
GET / PUT	`/api/settings`	模型配置（**不回传 key**，只回 `has_key`）
POST	`/api/settings/clear`	清除配置，回退 `.env`
GET	`/api/library`	资料库列表（含课程标签）
POST	`/api/library/upload`	上传 PDF（multipart，可带 `course`）
DELETE	`/api/library/{doc_id}`	删除文档与留底原件
POST	`/api/ask`	问答（非流式）
POST	`/api/ask/stream`	问答（SSE，事件：meta / token / done / not_found / error）
GET	`/api/page/{doc_id}/{page_no}`	原页 PNG（`dpi` 钳制在 50–300）
GET	`/api/course-names`	课程名列表（前端下拉用）
GET	`/api/semester`	周视图（可传 `?day=YYYY-MM-DD`）

方法	路径	说明
GET	`/api/semester/meta`	课表与作业条数、是否自定义
PUT	`/api/semester/courses`	导入课表（校验失败返回 422 并指名字段）
PUT	`/api/semester/assignments`	导入作业
POST	`/api/semester/reset`	恢复仓库自带示例数据

A.3 前端结构

```
frontend/src/
  App.tsx              主界面（问答 / 周视图 / 资料库 / 导入 / 设置）
  api.ts               接口封装，含超时分级、错误分类、SSE 解析
  types.ts             与后端返回结构一一对应
  components/cds/      来自 open-claude-ui 的组件（MIT，配色已替换）
  styles/cds-theme.css 183 个设计令牌
```